

March 25, 2025

1 Credit Card Approval Prediction

1.0.1 Project Overview:

A bank's credit card department is one of the top adopters of data science. A top focus for the bank has always been acquiring new credit card customers. Giving out credit cards without doing proper research or evaluating applicants' creditworthiness is quite risky. The credit card department has been using a data-driven system for credit assessment called Credit Scoring for many years, and the model is known as an application scorecard. A credit card application's cutoff value is determined using the application scorecard, which also aids in estimating the applicant's level of risk. This decision is made based on strategic priority at a given time. Customers must fill out a form, either physically or online, to apply for a credit card. The application data is used to evaluate the applicant's creditworthiness. The decision is made using the application data in addition to the Credit Bureau Score, such as the FICO Score in the US or the CIBIL Score in India, and other internal information on the applicants. Additionally, the banks are rapidly taking a lot of outside data into account to enhance the caliber of credit judgements.

1.0.2 Project Objective:

The main objective of this assignment is to minimize the risk and maximize the profit of the bank. Bank has to make a decision based on the applicant's profile to minimize the loss from the bank's perspective. Bank considers the applicant's over their nature of work, income range and family orientation details to take any decision to approve or reject a credit card application. The customer Credit card data contains many features and a classification approach to identify the credit worthiness of an applicant.

In this project we are utilizing the exploratory data analysis (EDA) as a data exploration technique to acquire knowledge, discover new relations, apply new methodologies and unravel patterns in data. It is important to apply the necessary rationale behind each step to address the main objective of the study.

So, The primary objective of this project is to develop a machine learning model for Credit Card Approval Prediction.

1.1 Feature Understanding

Dataset name: (Credit_Card.csv)

- Ind_ID: Client ID
- Gender: Gender information
- Car_owner: Having car or not

- Propert_owner: Having property or not
- Children: Count of children
- Annual_income: Annual income
- Type_Income: Income type
- Education: Education level
- Marital_status: Marital_status
- Housing_type: Living style
- Birthday_count: Use backward count from current day (0), -1 means yesterday.
- Employed_days: Start date of employment. Use backward count from current day (0). Positive value means, individual is currently unemployed.
- Mobile_phone: Any mobile phone
- Work_phone: Any work phone
- Phone: Any phone number
- EMAIL_ID: Any email ID
- Type_Occupation: Occupation
- Family_Members: Family size

Another data set (Credit_card_label.csv) contains two key pieces of information - ID: The joining key between application data and credit status data, same is Ind_ID - Label: 0 is application approved and 1 is application rejected.

1.1.1 Required Libraries

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.simplefilter('ignore')
pd.set_option('display.max_rows',None)
pd.set_option('display.max_columns',None)
```

1.1.2 Import Datasets

```
[2]: credit_card=pd.read_csv('Credit_card.csv')
credit_card_label=pd.read_csv('Credit_card_label.csv')
```

```
[3]: credit_card.head()
```

```
[3]:      Ind_ID  GENDER  Car_Owner  Propert_Owner  CHILDREN  Annual_income  \
0   5056149      M      Y      N      0      157500.0
1   5090386      F      N      Y      0      247500.0
2   5033628      F      N      Y      0      166500.0
3   5126108      F      N      Y      0      112500.0
4   5112599      F      N      Y      0      103500.0

      Type_Income      EDUCATION      Marital_status  \
```

0	Working	Secondary / secondary special	Married
1	Commercial associate	Higher education	Single / not married
2	State servant	Higher education	Married
3	State servant	Secondary / secondary special	Single / not married
4	Working	Secondary / secondary special	Married

	Housing_type	Birthday_count	Employed_days	Mobile_phone	Work_Phone \
0	House / apartment	-23424.0	-14887	1	0
1	House / apartment	-21537.0	-13735	1	0
2	House / apartment	-23599.0	-13382	1	0
3	House / apartment	-20577.0	-13010	1	1
4	House / apartment	-22466.0	-12870	1	1

	Phone	EMAIL_ID	Type_Occupation	Family_Members
0	0	0	Laborers	2
1	0	0	Core staff	1
2	0	0	NaN	2
3	0	0	Core staff	1
4	1	0	Managers	2

```
[4]: credit_card_label.head()
```

```
[4]:
```

	Ind_ID	label
0	5008827	1
1	5009744	1
2	5009746	1
3	5009749	1
4	5009752	1

1.1.3 Merging Both DataSets Using Pandas Merge Function

```
[5]: data=pd.merge(credit_card,credit_card_label,on='Ind_ID',how='inner')
```

```
[6]: df=data.copy()
```

```
[7]: df.head()
```

```
[7]:
```

	Ind_ID	GENDER	Car_Owner	Propert_Owner	CHILDREN	Annual_income \
0	5056149	M	Y	N	0	157500.0
1	5090386	F	N	Y	0	247500.0
2	5033628	F	N	Y	0	166500.0
3	5126108	F	N	Y	0	112500.0
4	5112599	F	N	Y	0	103500.0

	Type_Income	EDUCATION	Marital_status \
0	Working	Secondary / secondary special	Married
1	Commercial associate	Higher education	Single / not married

2	State servant	Higher education	Married
3	State servant	Secondary / secondary special	Single / not married
4	Working	Secondary / secondary special	Married

	Housing_type	Birthday_count	Employed_days	Mobile_phone	Work_Phone	\
0	House / apartment	-23424.0	-14887	1	0	
1	House / apartment	-21537.0	-13735	1	0	
2	House / apartment	-23599.0	-13382	1	0	
3	House / apartment	-20577.0	-13010	1	1	
4	House / apartment	-22466.0	-12870	1	1	

	Phone	EMAIL_ID	Type_Occupation	Family_Members	label
0	0	0	Laborers	2	0
1	0	0	Core staff	1	0
2	0	0	NaN	2	0
3	0	0	Core staff	1	0
4	1	0	Managers	2	0

1.1.4 Shape of DataFrame

```
[8]: print(f"Total Number of Rows in Dataset={df.shape[0]}")
      print(f'Total Number of Columns in Dataset={df.shape[1]}')
```

Total Number of Rows in Dataset=1548
Total Number of Columns in Dataset=19

- We can see that, dataset contains 1548 rows and 19 colmns.

1.2 Data Exploration

```
[9]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1548 entries, 0 to 1547
Data columns (total 19 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Ind_ID              1548 non-null   int64
1   GENDER              1541 non-null   object
2   Car_Owner           1548 non-null   object
3   Propert_Owner       1548 non-null   object
4   CHILDREN            1548 non-null   int64
5   Annual_income       1525 non-null   float64
6   Type_Income         1548 non-null   object
7   EDUCATION           1548 non-null   object
8   Marital_status      1548 non-null   object
```

```

9   Housing_type      1548 non-null   object
10  Birthday_count    1526 non-null   float64
11  Employed_days     1548 non-null   int64
12  Mobile_phone      1548 non-null   int64
13  Work_Phone        1548 non-null   int64
14  Phone             1548 non-null   int64
15  EMAIL_ID          1548 non-null   int64
16  Type_Occupation   1060 non-null   object
17  Family_Members    1548 non-null   int64
18  label             1548 non-null   int64
dtypes: float64(2), int64(9), object(8)
memory usage: 229.9+ KB

```

We can gather valuable Information about the dataset.

- Dataset contains 1548 entries(Rows) and there are 19 columns in the Dataset.
- Out of 19 columns, 11 columns are Numerical Columns and 8 Columns are Categorical Columns.
- Several columns are having missing value including GENDER, Annual_income, Birthday_count and Type_Occupation.

1.3 Splitting columns by data types

1.3.1 Categorical Columns

```

[10]: categorical_columns=df.select_dtypes(include='object').columns
      for i in categorical_columns:
          print(i)

```

```

GENDER
Car_Owner
Propert_Owner
Type_Income
EDUCATION
Marital_status
Housing_type
Type_Occupation

```

1.3.2 Numerical_Columns

```

[11]: numerical_columns=df.select_dtypes(include='number').columns
      for i in numerical_columns:
          print(i)

```

```

Ind_ID
CHILDREN
Annual_income
Birthday_count
Employed_days

```

```
Mobile_phone
Work_Phone
Phone
EMAIL_ID
Family_Members
label
```

1.4 Checking Null Values In Dataset

```
[12]: df.isnull().sum()/len(df)*100
```

```
[12]: Ind_ID          0.000000
      GENDER         0.452196
      Car_Owner      0.000000
      Propert_Owner  0.000000
      CHILDREN       0.000000
      Annual_income  1.485788
      Type_Income    0.000000
      EDUCATION      0.000000
      Marital_status 0.000000
      Housing_type   0.000000
      Birthday_count 1.421189
      Employed_days  0.000000
      Mobile_phone   0.000000
      Work_Phone     0.000000
      Phone          0.000000
      EMAIL_ID       0.000000
      Type_Occupation 31.524548
      Family_Members 0.000000
      label          0.000000
      dtype: float64
```

Here we can see

- Gender: 0.45% missing values
- Annual_income: 1.49% missing values
- Birthday_count: 1.42% missing values
- Type_Occupation: 31.52% missing values

1.5 Drop Unnecessery Columns

- The Features Mobile_phone, Work_Phone, Phone, EMAIL_ID are present in the dataset but these columns are unnecessary for data analysis. Drop these unnecessary columns.
- Type_Occupation contains 31.52% nulls values thats why we consider removing it.

```
[13]: df.drop(columns=["Mobile_phone", "Work_Phone", "Phone", "EMAIL_ID", 'Type_Occupation'],axis=1,inplace=True)
```

```
[14]: df.columns
```

```
[14]: Index(['Ind_ID', 'GENDER', 'Car_Owner', 'Propert_Owner', 'CHILDREN',
        'Annual_income', 'Type_Income', 'EDUCATION', 'Marital_status',
        'Housing_type', 'Birthday_count', 'Employed_days', 'Family_Members',
        'label'],
        dtype='object')
```

1.5.1 Feature Engineering

Calculate the approx age of customers using Birthday count:

```
[15]: import math
Age=[]
for i in df['Birthday_count']:
    if not math.isnan(i):
        a=i/365
        Age.append(round(abs(a)))
    else:
        Age.append(np.nan)
df['Age']=Age
```

1.5.2 Creating an 'Emmloyed_Status' Feature from 'Employed_days'

```
[16]: Employed_Status=[]
for i in df['Employed_days']:
    if i<0:
        Employed_Status.append('Employed')
    else:
        Employed_Status.append('Unemployed')
df['Employed_Status']=Employed_Status
```

```
[17]: df.drop(columns=['Birthday_count','Employed_days'],axis=1,inplace=True)
```

```
[18]: df.rename(columns={'label':'Approved_status'},inplace=True)
```

```
[19]: df.head()
```

```
[19]:   Ind_ID  GENDER  Car_Owner  Propert_Owner  CHILDREN  Annual_income  \
0  5056149      M          Y              N          0      157500.0
1  5090386      F          N              Y          0      247500.0
2  5033628      F          N              Y          0      166500.0
3  5126108      F          N              Y          0      112500.0
4  5112599      F          N              Y          0      103500.0
```

	Type_Income	EDUCATION	Marital_status \
0	Working	Secondary / secondary special	Married
1	Commercial associate	Higher education	Single / not married
2	State servant	Higher education	Married
3	State servant	Secondary / secondary special	Single / not married
4	Working	Secondary / secondary special	Married

	Housing_type	Family_Members	Approved_status	Age	Employed_Status
0	House / apartment	2	0	64.0	Employed
1	House / apartment	1	0	59.0	Employed
2	House / apartment	2	0	65.0	Employed
3	House / apartment	1	0	56.0	Employed
4	House / apartment	2	0	62.0	Employed

- Now we have 14 Features to Analysis

1.6 Overall Statistics about the Dataset

```
[20]: df.describe()
```

```
[20]:
```

	Ind_ID	CHILDREN	Annual_income	Family_Members \
count	1.548000e+03	1548.000000	1.525000e+03	1548.000000
mean	5.078920e+06	0.412791	1.913993e+05	2.161499
std	4.171759e+04	0.776691	1.132530e+05	0.947772
min	5.008827e+06	0.000000	3.375000e+04	1.000000
25%	5.045070e+06	0.000000	1.215000e+05	2.000000
50%	5.078842e+06	0.000000	1.665000e+05	2.000000
75%	5.115673e+06	1.000000	2.250000e+05	3.000000
max	5.150412e+06	14.000000	1.575000e+06	15.000000

	Approved_status	Age
count	1548.000000	1526.000000
mean	0.113049	43.952818
std	0.316755	11.603295
min	0.000000	21.000000
25%	0.000000	34.000000
50%	0.000000	43.000000
75%	0.000000	54.000000
max	1.000000	68.000000

1.7 Data Summary Report

- The average income is approximetly 1,91,399.30,with a standard deviation of 1,13,253.0, suggesting a wide income distribution.
- The minimum and maximum Annual_income are 33,750.00 and 1,57,500.0.
- The Range of age is between 21 years to 68 years and average age of customers is approx 44 years.


```
[21]: df.describe(include='object')
```

```
[21]:      GENDER Car_Owner Propert_Owner Type_Income \
count      1541      1548      1548      1548
unique        2        2        2        4
top          F        N        Y    Working
freq        973      924      1010      798

      EDUCATION Marital_status      Housing_type \
count              1548      1548      1548
unique              5        5        6
top    Secondary / secondary special    Married    House / apartment
freq              1031      1049      1380

      Employed_Status
count              1548
unique              2
top          Employed
freq              1287
```

1.7.1 Data Summary

- In Dataset most of the customers are not having car. means, they are not car owners.
- In dataset most of the customers are married and their education is Secondary/secondary special.
- There is most of the customers are Employed, Working and living in House/apartment.
- And, Most of the customers are Property owner.*

1.7.2 Check Unique Values in Categorical Columns

```
[22]: categorical_column=df.select_dtypes(include='object').columns
for i in categorical_column:
    print(f"Unique values in {i} column")
    print(df[i].unique())
    print('-'*50)
```

```
Unique values in GENDER column
['M' 'F' nan]
```

```
-----
Unique values in Car_Owner column
['Y' 'N']
```

```
-----
Unique values in Propert_Owner column
['N' 'Y']
-----
```

```

Unique values in Type_Income column
['Working' 'Commercial associate' 'State servant' 'Pensioner']
-----

Unique values in EDUCATION column
['Secondary / secondary special' 'Higher education' 'Incomplete higher'
 'Academic degree' 'Lower secondary']
-----

Unique values in Marital_status column
['Married' 'Single / not married' 'Separated' 'Civil marriage' 'Widow']
-----

Unique values in Housing_type column
['House / apartment' 'Municipal apartment' 'With parents'
 'Co-op apartment' 'Office apartment' 'Rented apartment']
-----

Unique values in Employed_Status column
['Employed' 'Unemployed']
-----

```

Gender Column is having nulls values.

1.7.3 Modifying categories in categorical columns

```

[23]: Marital_status_mapping={'Married':'Married',
                             'Separated':'Separated',
                             'Civil marriage':'Civil marriage',
                             'Single / not married':'not married',
                             'Widow':'Widow'}
df['Marital_status']=df['Marital_status'].map(Marital_status_mapping)

df['Marital_status'].unique()

```

```

[23]: array(['Married', 'not married', 'Separated', 'Civil marriage', 'Widow'],
          dtype=object)

```

```

[24]: EDUCATION_mapping={'Higher education':'Higher Education',
                        'Secondary / secondary special':'Secondary Education',
                        'Incomplete higher':'Incomplete Higher',
                        'Lower secondary':'Lower Secondary',
                        'Academic degree':'Academic Degree'}
df['EDUCATION']=df['EDUCATION'].map(EDUCATION_mapping)

df['EDUCATION'].unique()

```

```

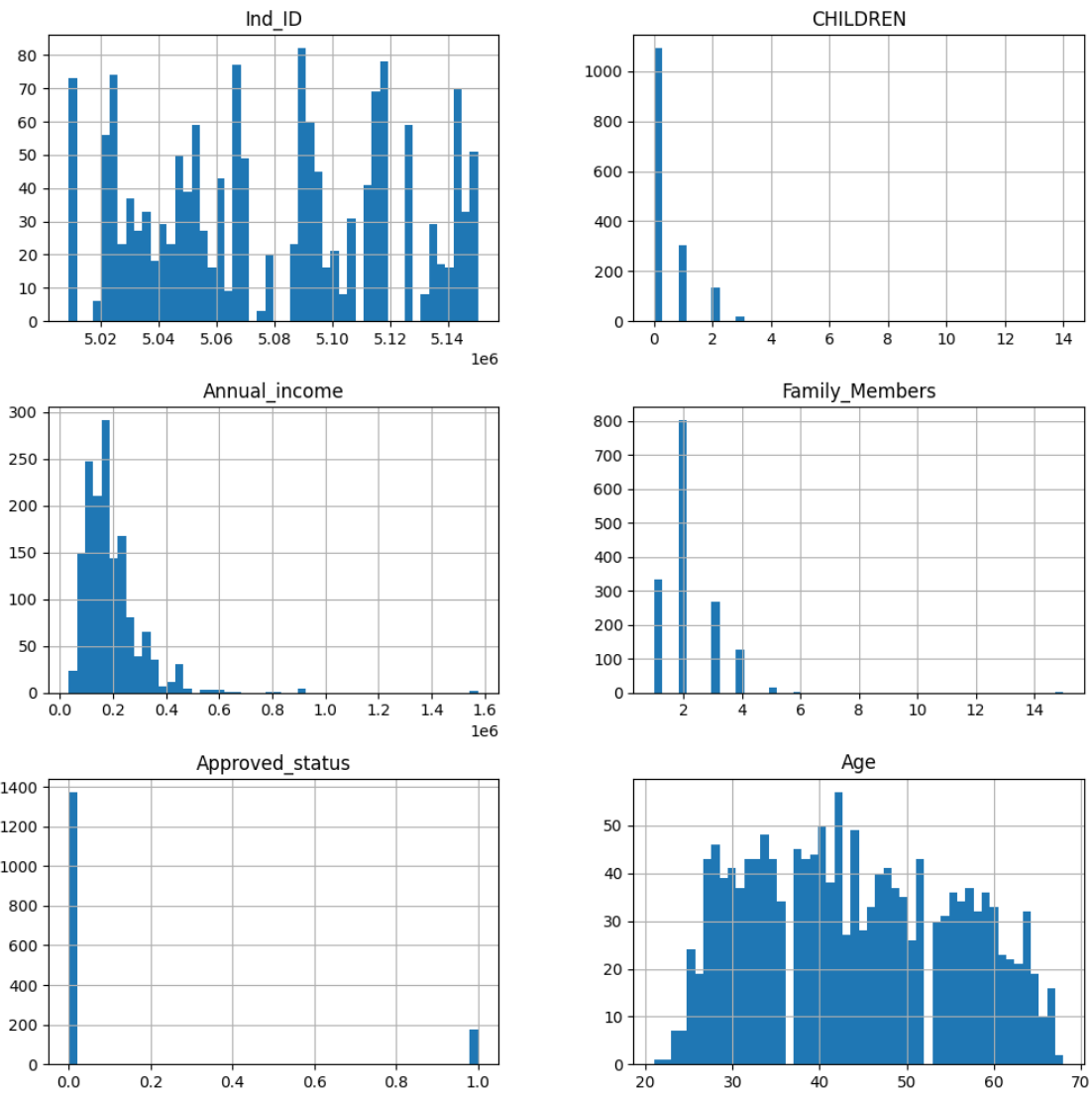
[24]: array(['Secondary Education', 'Higher Education', 'Incomplete Higher',
            'Academic Degree', 'Lower Secondary'], dtype=object)

```

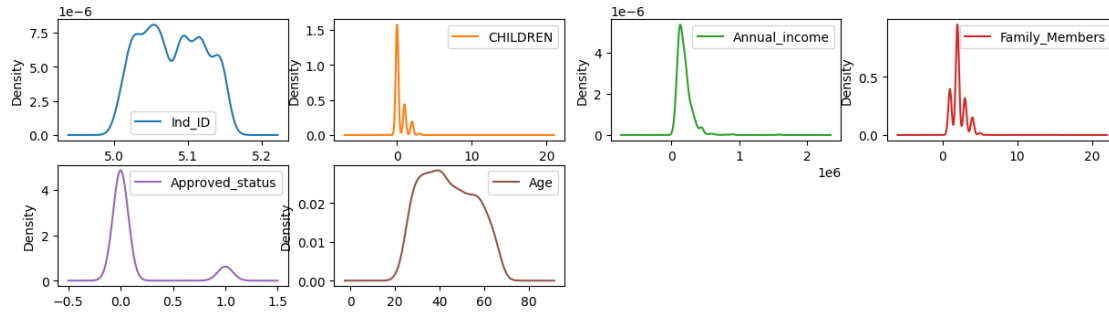
1.7.4 Visualizing the Data for Better Understanding

Distribution of Numerical Variables

```
[25]: df.hist(bins=50,figsize=(12,12))  
plt.show()
```



```
[26]: df.plot(kind='density',subplots=True,figsize=(15,10),sharex=False,layout=(5,4))  
plt.show()
```



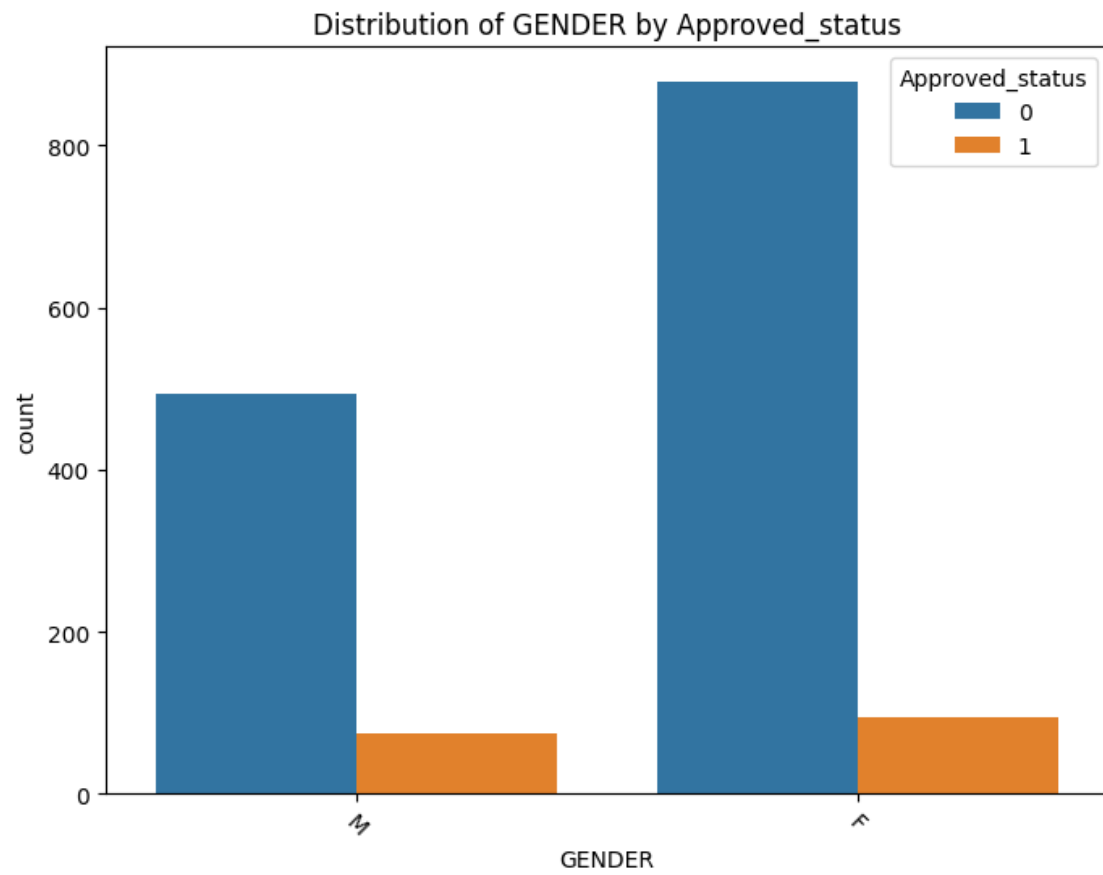
- With above visualization we can say that there is zero children customers are more and very less customers has 3 children.
- We can see that, max of customers are belongs to less than 40,000 annual income and very less customers are belongs to more than 40,000 annual income.
- we can see that,max of customer's family members are couples and very less customer,s family members are above 4.
- With above visualization, we can see most of customers are belongs to range of 21 to 68 years age.

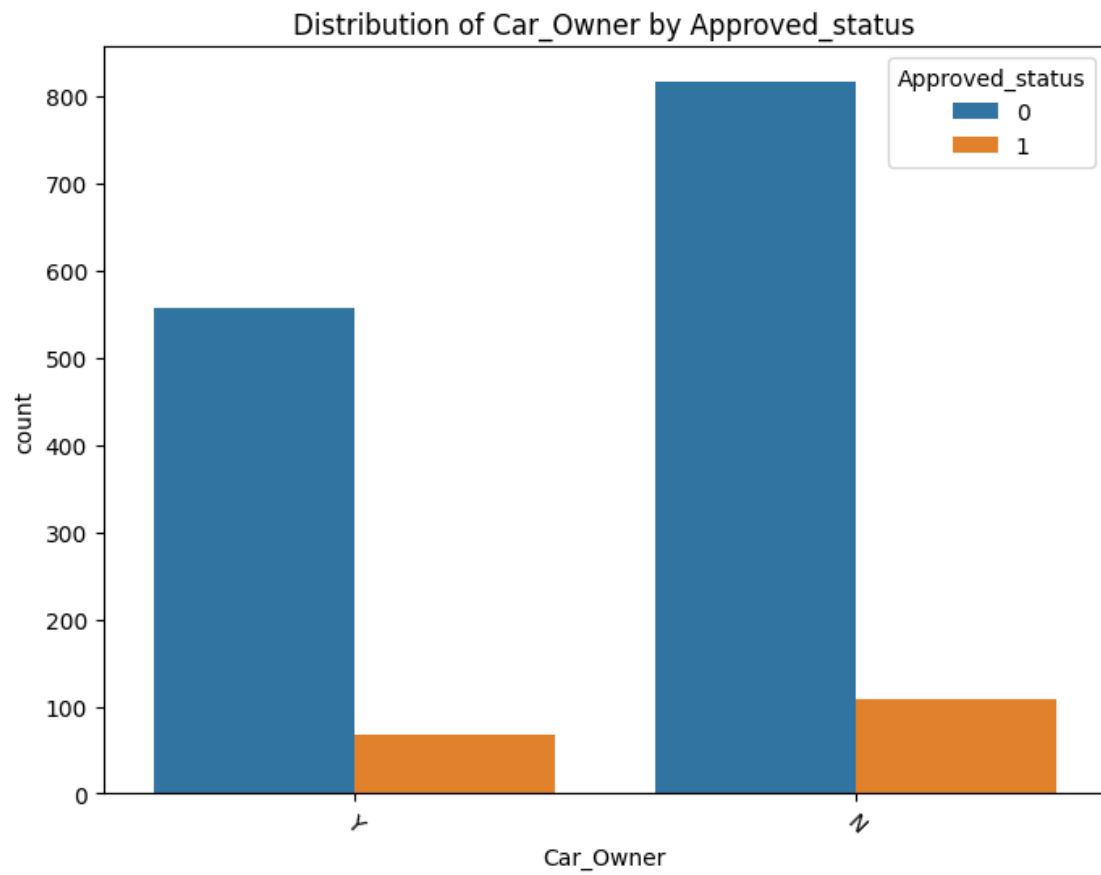
```
[27]: numerical_columns=[]
      for i in df.select_dtypes(include='number'):
          numerical_columns.append(i)
```

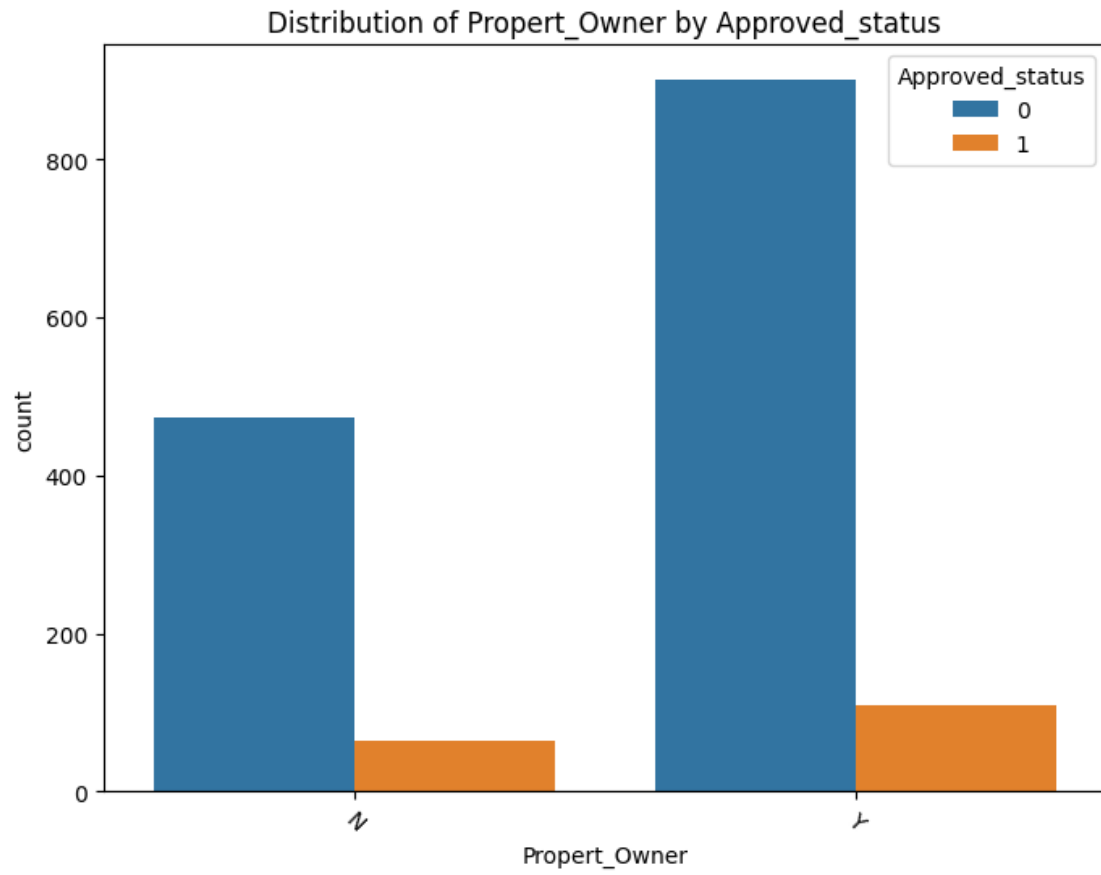
```
[28]: numerical_columns
```

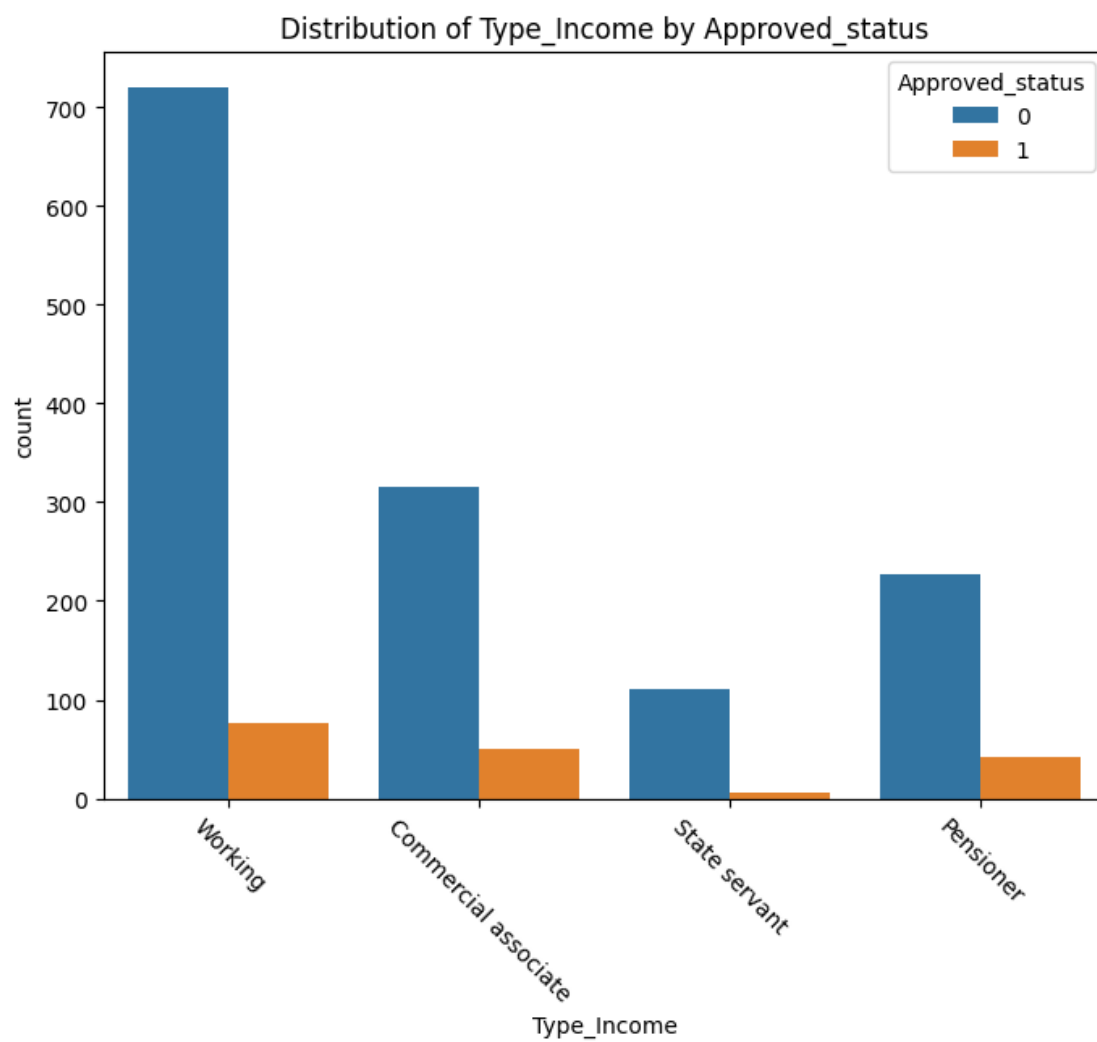
```
[28]: ['Ind_ID',
       'CHILDREN',
       'Annual_income',
       'Family_Members',
       'Approved_status',
       'Age']
```

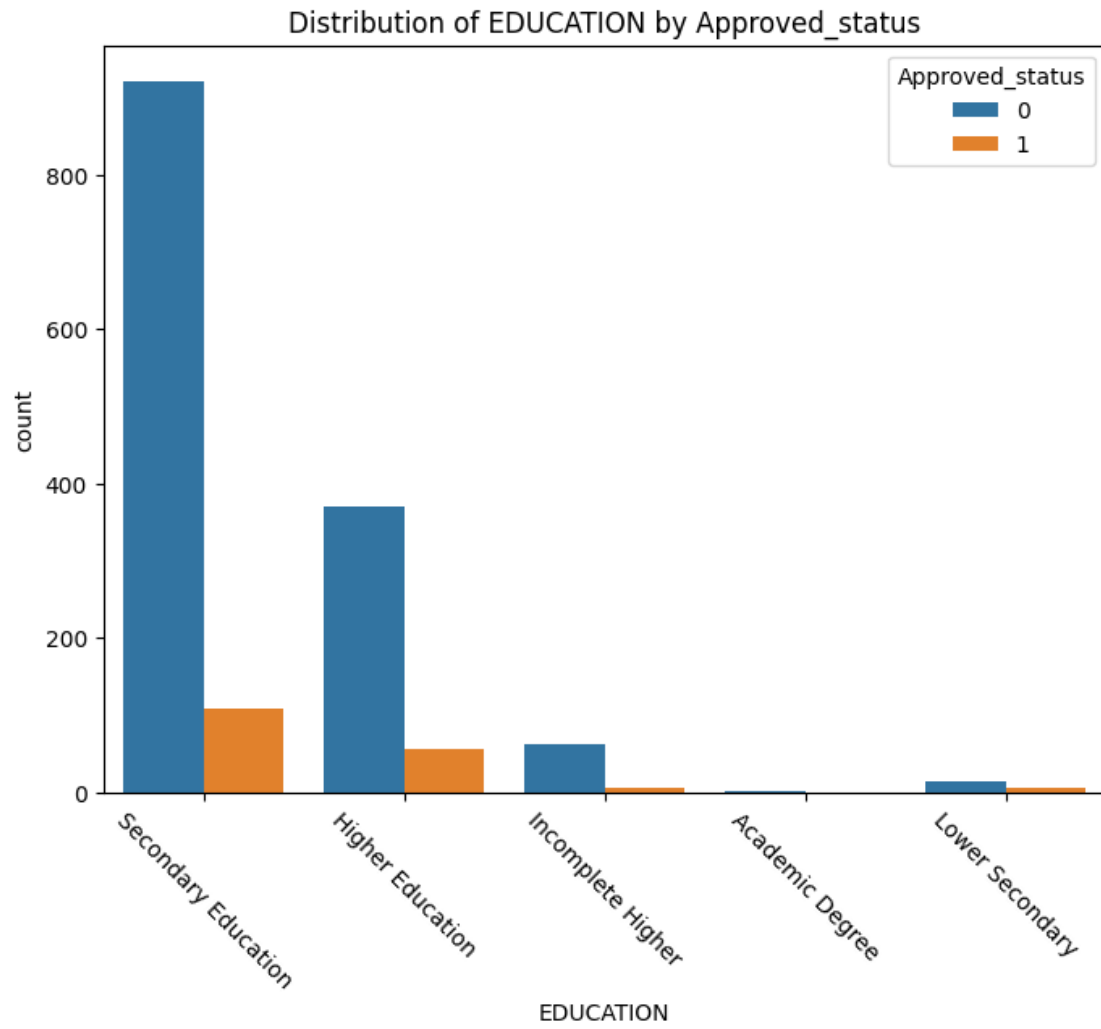
```
[29]: categorical_features=['GENDER', 'Car_Owner', 'Propert_Owner', 'Type_Income',
                             'EDUCATION', 'Marital_status', 'Housing_type', 'Employed_Status']
      for feature in categorical_features:
          plt.figure(figsize=(8,6))
          sns.countplot(x=feature,data=df,hue='Approved_status')
          plt.xticks(rotation=-45)
          plt.title(f'Distribution of {feature} by Approved_status')
          plt.show()
```

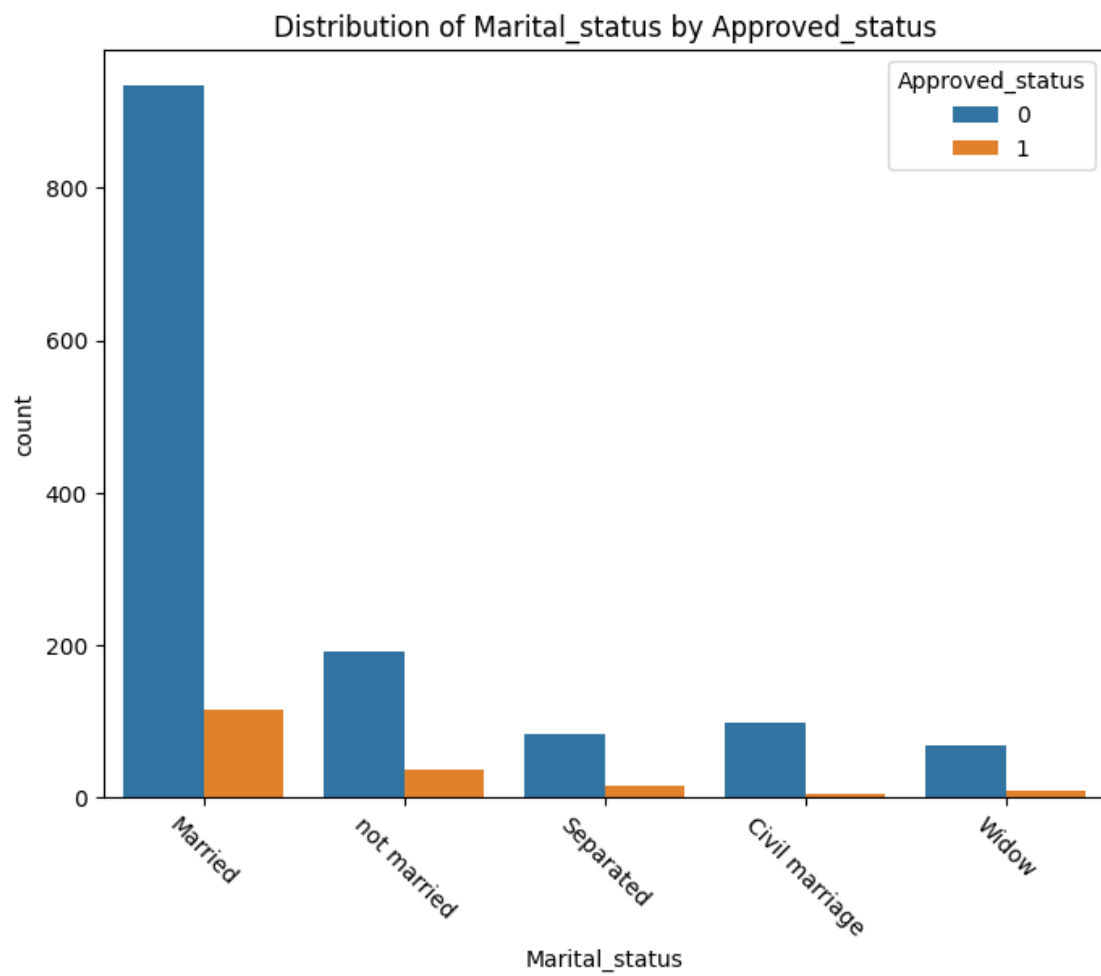


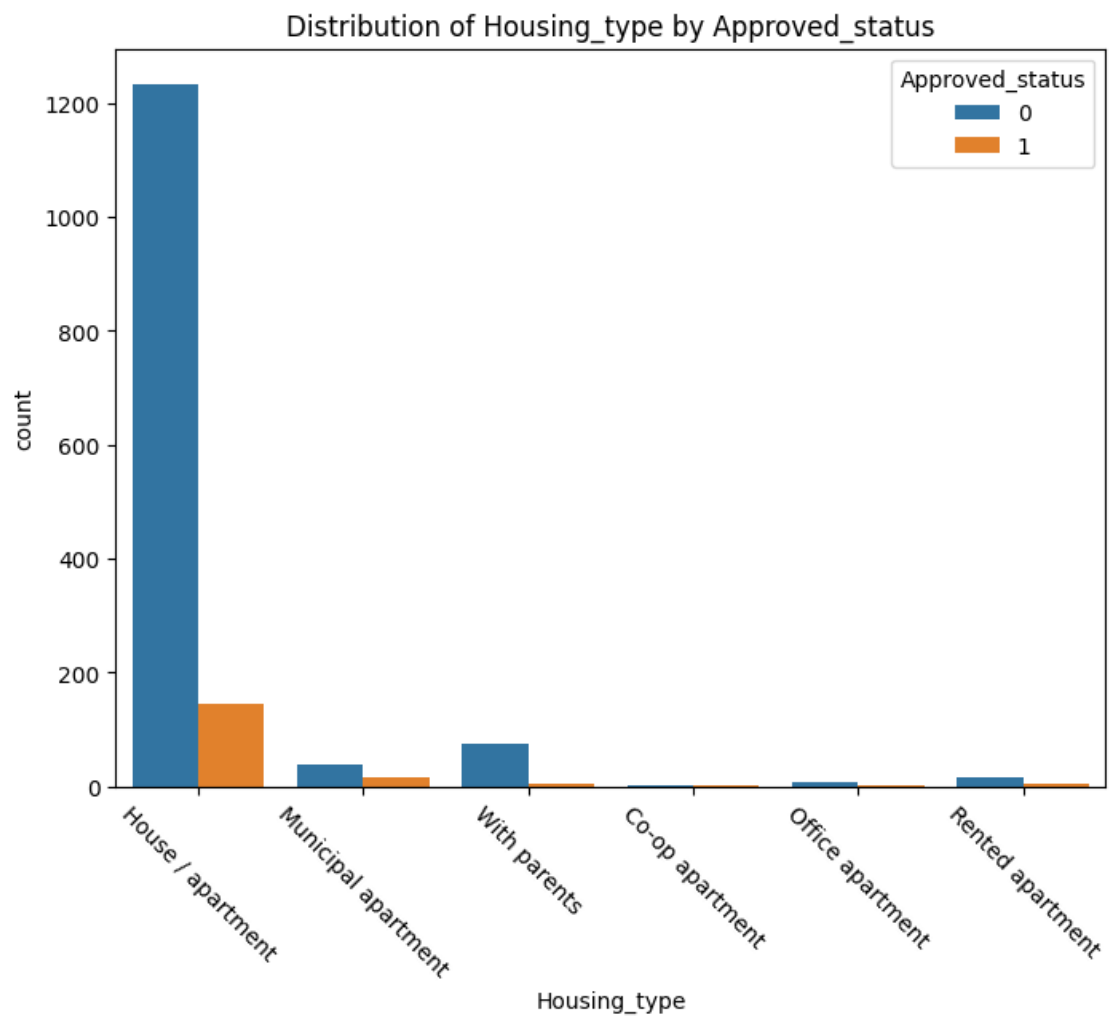


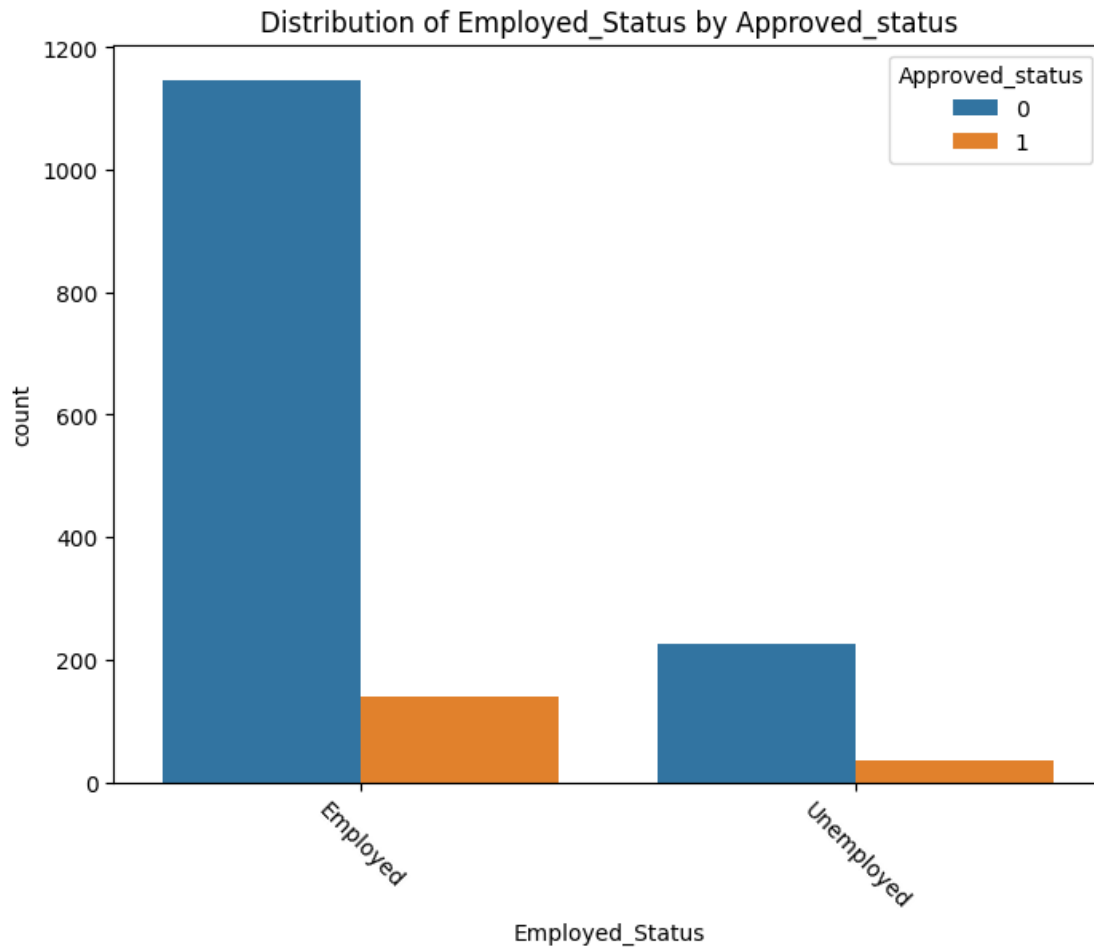








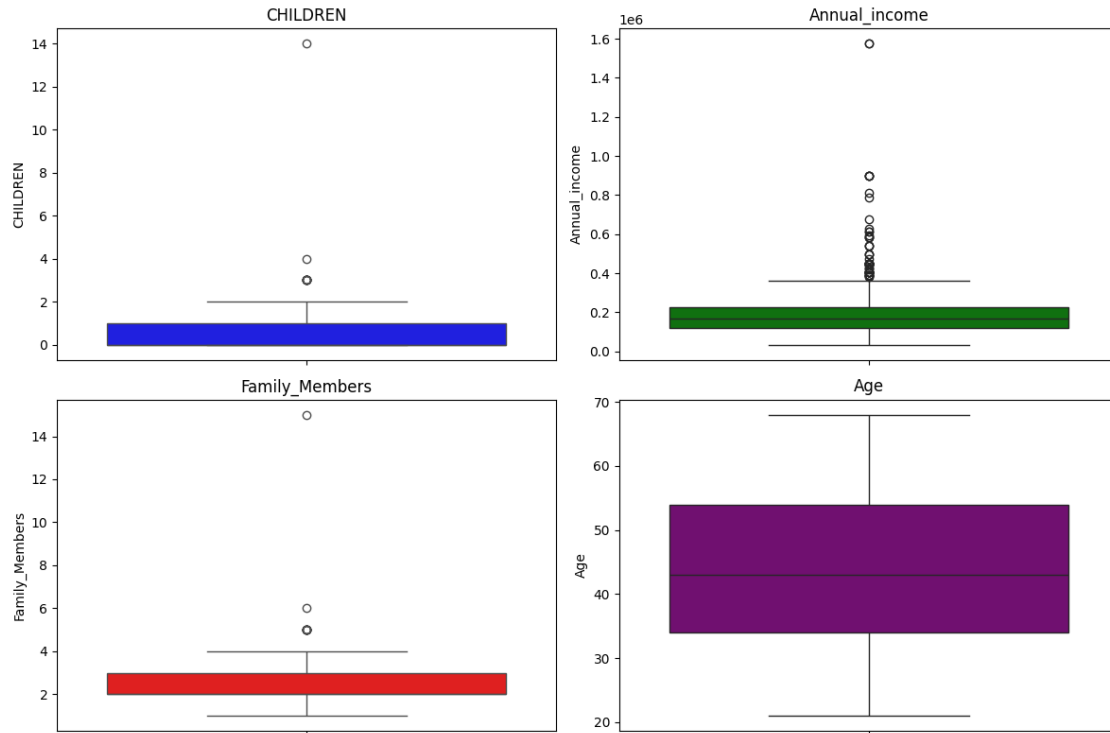




1.7.5 Boxplot to check Outliers in dataset

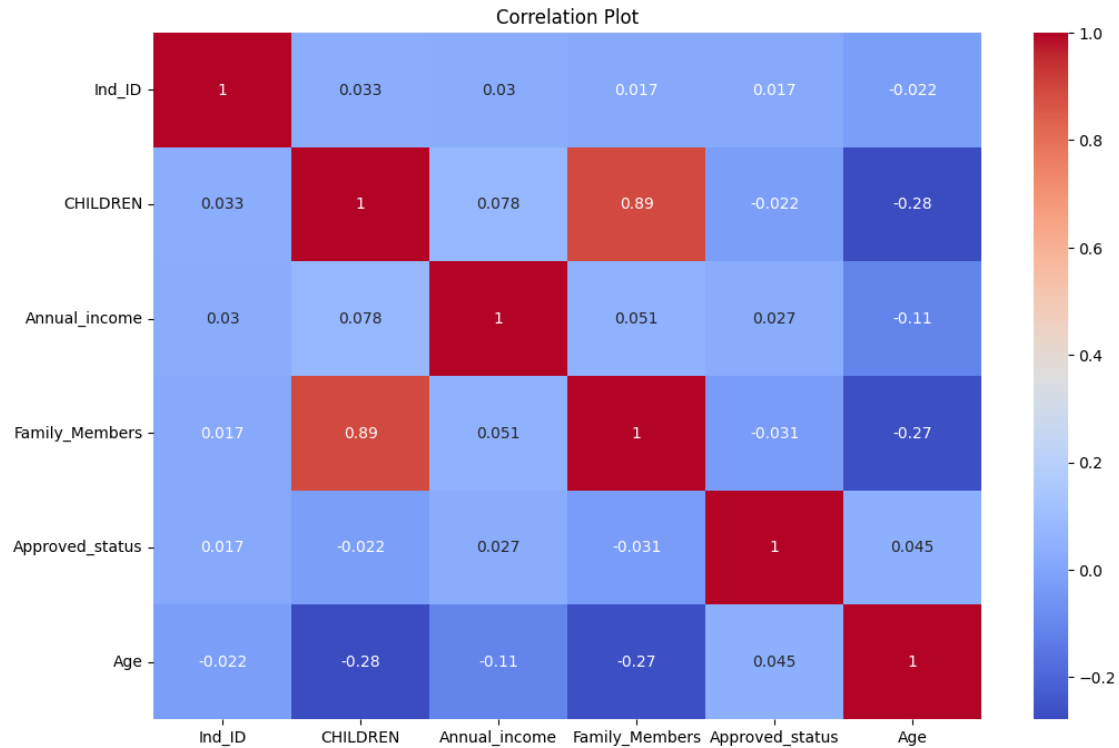
```
[30]: columns_to_plot=['CHILDREN', 'Annual_income','Family_Members','Age']
      colors=['blue','green','red','purple']
      fig,axes=plt.subplots(nrows=2,ncols=2,figsize=(12,8))
      for i,column in enumerate(columns_to_plot):
          sns.boxplot(data=df[column],ax=axes[i//2,i%2],palette=[colors[i]])
          axes[i//2,i%2].set_title(column)

      plt.tight_layout()
      plt.show()
```



1.7.6 So the dataset contains outliers in features like **CHILDREN**, **Annual income** and **Family_Members**

```
[31]: corr_matrix=df.corr(numeric_only=True)
plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix,annot=True,fmt='.
↪2g',cmap='coolwarm',linewidths=0,linecolor='black',cbar=True)
plt.title('Correlation Plot')
plt.show()
```



1.7.7 It has shown there is a strong correlation between Family_Members and Children.

1.8 Data Preprocessing

1.8.1 Handling missing values

```
[32]: df.isnull().sum()
```

```
[32]: Ind_ID          0
      GENDER          7
      Car_Owner       0
      Propert_Owner   0
      CHILDREN        0
      Annual_income   23
      Type_Income     0
      EDUCATION       0
      Marital_status  0
      Housing_type    0
      Family_Members  0
      Approved_status 0
      Age            22
      Employed_Status 0
      dtype: int64
```

```
[33]: df['Annual_income']=df['Annual_income'].fillna(df['Annual_income'].median())
df['Age']=df['Age'].fillna(df['Age'].mean())
df['GENDER']=df['GENDER'].fillna(df['GENDER'].mode()[0])
```

Handle Outlier in Dataset

```
[34]: Q1=df['Annual_income'].quantile(0.25)
Q3=df['Annual_income'].quantile(0.75)

IQR=Q3-Q1
Lower_limit=Q1-(1.5*IQR)
Upper_limit=Q3+(1.5*IQR)

df['Annual_income']=df['Annual_income'].clip(Lower_limit,Upper_limit)
```

```
[35]: df=df.drop(['Ind_ID'],axis=1)
```

```
[36]: df.columns
```

```
[36]: Index(['GENDER', 'Car_Owner', 'Propert_Owner', 'CHILDREN', 'Annual_income',
          'Type_Income', 'EDUCATION', 'Marital_status', 'Housing_type',
          'Family_Members', 'Approved_status', 'Age', 'Employed_Status'],
          dtype='object')
```

```
[37]: categorical_column
```

```
[37]: Index(['GENDER', 'Car_Owner', 'Propert_Owner', 'Type_Income', 'EDUCATION',
          'Marital_status', 'Housing_type', 'Employed_Status'],
          dtype='object')
```

1.9 Dummy Encoding

```
[38]: categorical_column
```

```
[38]: Index(['GENDER', 'Car_Owner', 'Propert_Owner', 'Type_Income', 'EDUCATION',
          'Marital_status', 'Housing_type', 'Employed_Status'],
          dtype='object')
```

```
[39]: XX=pd.get_dummies(df,columns=categorical_column,drop_first=True)
```

1.10 Seperate Independent Variable X and Dependent Variable Y

```
[40]: X=XX.drop('Approved_status',axis=1)
y=df['Approved_status']
```

1.11 Split the data in training and testing sets

```
[41]: from sklearn.model_selection import train_test_split
```

```
[42]: X_train,X_test,y_train,y_test=train_test_split(X,y,train_size=0.  
↳8,random_state=17)
```

```
[43]: print(len(X_train))  
print(len(X_test))
```

1238

310

1.12 Feature Scaling

```
[44]: from sklearn.preprocessing import StandardScaler  
sc=StandardScaler()  
X_train=sc.fit_transform(X_train)  
x_test=sc.transform(X_test)
```

```
[45]: print(len(X_train))  
print(len((X_test)))
```

1238

310

1.13 Key Findings

1.13.1 Income Type(Working)

- Customers with a 'Working' Income is more than other Income types and their applications approvals are more than other type .

1.13.2 Marital Status(Married)

- Married customers are more than other type of Marital status and their applications approval are more than other marital status types.

1.13.3 Housing Type(House/Apartment)

- Most of the customers live in House/Apartment and their application approvals are more than other housing types.

1.14 Final Conclusion

- When considering all three types together, individual who are working, married and live in House/apartment type there is a higher probability of having their application approved.

1.15 Modeling

1.16 1. Logistic Regression

```
[46]: from sklearn.linear_model import LogisticRegression
logistic = LogisticRegression()
logistic.fit(X_train,y_train)

## Prediction
log_pred_train = logistic.predict(X_train)
log_pred_test = logistic.predict(X_test)

## Evaluation
from sklearn.metrics import accuracy_score
log_train_accuracy = accuracy_score(y_train,log_pred_train)*100
log_test_accuracy = accuracy_score(y_test,log_pred_test)*100

from sklearn.model_selection import cross_val_score
log_cv = cross_val_score(logistic,X_test,y_test,cv=5,scoring='accuracy').mean()
↳* 100

print(f"Train accuracy score: {round(log_train_accuracy,2)}%")
print(f"Test accuracy score: {round(log_test_accuracy,2)}%")
print(f"Cross validation score: {round(log_cv,2)}%")
```

Train accuracy score: 89.98%

Test accuracy score: 13.23%

Cross validation score: 86.45%

1.17 2. Support Vector Machine hyperparameter tuning

```
[47]: from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

sv_model = SVC()

param_grid = {'C': [0.1, 1, 10, 100],
              "gamma": [1, 0.1, 0.01, 0.001],
              "kernel": ['rbf', 'sigmoid', 'poly']}

grid_search = GridSearchCV(estimator = sv_model, param_grid=param_grid,
↳cv=5,scoring='accuracy')
grid_search.fit(X_train,y_train)
grid_search.best_params_
```

```
[47]: {'C': 1, 'gamma': 1, 'kernel': 'rbf'}
```

1.18 2. Support Vector Machine

```
[48]: from sklearn.svm import SVC
sv_model = SVC(C= 1, gamma= 1, kernel='rbf')
sv_model.fit(X_train,y_train)

# prediction
sv_pred_train = sv_model.predict(X_train)
sv_pred_test = sv_model.predict(X_test)

# Evaluation
from sklearn.metrics import accuracy_score,confusion_matrix
svm_train_accuracy = accuracy_score(y_train,sv_pred_train)*100
svm_test_accuracy = accuracy_score(y_test,sv_pred_test)*100

from sklearn.model_selection import cross_val_score
svm_cv = cross_val_score(sv_model,X_test,y_test,cv=5,scoring='accuracy').
    ↪mean()*100

print(f"Train accuracy score: {round(svm_train_accuracy,2)}%")
print(f"Test accuracy score: {round(svm_test_accuracy,2)}%")
print(f"Cross validation score: {round(svm_cv,2)}%')
print('confusion matrix:')
print(confusion_matrix(y_test,sv_pred_test))
```

Train accuracy score: 94.99%

Test accuracy score: 86.77%

Cross validation score: 88.06%

confusion matrix:

```
[[269   0]
 [ 41   0]]
```

1.19 3. Decision Tree Hyperparameter Tuning

```
[49]: from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

estimator = DecisionTreeClassifier(random_state=15)
param_grid = {'criterion':['gini','entropy'],
              'max_depth':list(range(1,16))}

grid = GridSearchCV(estimator = estimator, param_grid = param_grid, cv =5 ,
    ↪scoring='accuracy')
grid.fit(X_train,y_train)
grid.best_params_
```

```
[49]: {'criterion': 'entropy', 'max_depth': 3}
```

```
[50]: from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier(criterion = 'entropy', max_depth = 3)
dt_model.fit(X_train,y_train)

## Prediction
dt_pred_train = dt_model.predict(X_train)
dt_pred_test = dt_model.predict(X_test)

from sklearn.metrics import accuracy_score,confusion_matrix
dt_train_accuracy = accuracy_score(y_train,dt_pred_train)*100
dt_test_accuracy = accuracy_score(y_test,dt_pred_test)*100

from sklearn.model_selection import cross_val_score
dt_cv = cross_val_score(dt_model,X_test,y_test,cv=5,scoring='accuracy').
    ↪mean()*100

print(f"Train accuracy score: {round(dt_train_accuracy,2)}%")
print(f"Test accuracy score: {round(dt_test_accuracy,2)}%")
print(f'Cross validation score: {round(dt_cv,2)}%')
print('confusion matrix:')
print(confusion_matrix(y_test,dt_pred_test))
```

Train accuracy score: 89.58%

Test accuracy score: 86.77%

Cross validation score: 85.81%

confusion matrix:

```
[[269   0]
 [ 41   0]]
```

1.20 4. Random Forest Hyperparameter Tuning

```
[51]: from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
estimator = RandomForestClassifier()

params = {'n_estimators':[100,200, 300, 400,500],
          'max_depth':[None,10,20,30,40,50],
          "min_samples_split":[2,5,10],
          "min_samples_leaf":[1,2,4],
          "bootstrap":[True,False]}

grid_search = GridSearchCV(estimator = estimator, param_grid = params, cv=5,
    ↪scoring='accuracy')
grid_search.fit(X_train,y_train)
grid_search.best_params_
```

```
[51]: {'bootstrap': True,
      'max_depth': 40,
      'min_samples_leaf': 1,
      'min_samples_split': 2,
      'n_estimators': 100}
```

```
[52]: from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(n_estimators=100, max_depth=30,
    ↪min_samples_split=2, min_samples_leaf=1, bootstrap=True)
rf_model.fit(X_train,y_train)

## Prediction
rf_pred_train = rf_model.predict(X_train)
rf_pred_test = rf_model.predict(X_test)

from sklearn.metrics import accuracy_score,confusion_matrix
rf_train_accuracy = accuracy_score(y_train,rf_pred_train)*100
rf_test_accuracy = accuracy_score(y_test,rf_pred_test)*100

from sklearn.model_selection import cross_val_score
rf_cv = cross_val_score(rf_model,X_test,y_test,cv=5,scoring='accuracy').
    ↪mean()*100

print(f"Train accuracy score: {round(rf_train_accuracy,2)}%")
print(f"Test accuracy score: {round(rf_test_accuracy,2)}%")
print(f"Cross validation score: {round(rf_cv,2)}%")
print('confusion matrix:')
print(confusion_matrix(y_test,rf_pred_test))
```

Train accuracy score: 99.11%

Test accuracy score: 86.77%

Cross validation score: 87.74%

confusion matrix:

```
[[269   0]
 [ 41   0]]
```

1.21 5. XGBoost Hyperparameter Tuning

```
[55]: #!pip install xgboost
```

```
[56]: from sklearn.model_selection import GridSearchCV
from xgboost import XGBClassifier

estimator = XGBClassifier()
params = {'n_estimator': [100,200,300,400,500],
          'max_depth': [None,10,20,30,40,50],
          'learning_rate': [0.1,0.01,0.001]}
```

```

grid_search = GridSearchCV(estimator=estimator, param_grid=params,cv=5,
    ↪scoring='accuracy')
grid_search.fit(X_train,y_train)
grid_search.best_params_

```

```
[56]: {'learning_rate': 0.1, 'max_depth': 20, 'n_estimator': 100}
```

```

[57]: from sklearn.ensemble import RandomForestClassifier
xgb_model = XGBClassifier(learning_rate=0.1, n_estimators=100, max_depth=20)
xgb_model.fit(X_train,y_train)

## Prediction
xgb_pred_train = xgb_model.predict(X_train)
xgb_pred_test = xgb_model.predict(X_test)

from sklearn.metrics import accuracy_score,confusion_matrix
xgb_train_accuracy = accuracy_score(y_train,xgb_pred_train)*100
xgb_test_accuracy = accuracy_score(y_test,xgb_pred_test)*100

from sklearn.model_selection import cross_val_score
xgb_cv = cross_val_score(xgb_model,X_test,y_test,cv=5,scoring='accuracy').
    ↪mean()*100

print(f"Train accuracy score: {round(xgb_train_accuracy,2)}%")
print(f"Test accuracy score: {round(xgb_test_accuracy,2)}%")
print(f"Cross validation score: {round(xgb_cv,2)}%")
print('confusion matrix:')
print(confusion_matrix(y_test,xgb_pred_test))

```

Train accuracy score: 99.03%

Test accuracy score: 86.13%

Cross validation score: 85.48%

confusion matrix:

```

[[267   2]
 [ 41   0]]

```

2 Artificial Neural Network(ANN)

```
[59]: #!pip install optuna
```

```

[64]: from keras.models import Sequential
from keras.layers import Dense, Dropout
from keras.optimizers import Adam, RMSprop, SGD
from keras.callbacks import EarlyStopping
import optuna

```

```
[65]: def objective(trial):
    # Hyperparameters to optimize
    num_layers = trial.suggest_int('num_layers', 1, 7)
    units = trial.suggest_int('units', 32, 128, step=16)
    activation_function = trial.suggest_categorical('activation_function',
    ↪['relu', 'leaky_relu', 'sigmoid', 'tanh'])
    learning_rate = trial.suggest_float('learning_rate', 1e-5, 1e-1, log=True)
    batch_size = trial.suggest_int('batch_size', 32, 128, step=32)
    optimizer = trial.suggest_categorical('optimizer', ['adam', 'sgd',
    ↪'rmsprop'])
    epochs = trial.suggest_int('epochs', 10, 100)
    dropout_rate = trial.suggest_float('dropout_rate', 0.1, 0.5)

    # Create model
    model = Sequential()

    # First hidden layer
    model.add(Dense(units=units, activation=activation_function,
    ↪input_dim=X_train.shape[1]))
    model.add(Dropout(dropout_rate))

    # Add additional layers
    for _ in range(num_layers - 1):
        model.add(Dense(units=units, activation=activation_function))
        model.add(Dropout(dropout_rate))

    # Output layer
    model.add(Dense(1, activation="sigmoid"))

    # Select optimizer
    if optimizer == "adam":
        optimizer_instance = Adam(learning_rate=learning_rate)
    elif optimizer == "sgd":
        optimizer_instance = SGD(learning_rate=learning_rate)
    elif optimizer == "rmsprop":
        optimizer_instance = RMSprop(learning_rate=learning_rate)

    # Compile the model
    model.compile(optimizer=optimizer_instance, loss="binary_crossentropy",
    ↪metrics=["accuracy"])

    # Train the model
    history = model.fit(X_train, y_train, epochs=epochs, batch_size=batch_size,
    ↪validation_data=(X_test, y_test), verbose=0)

    # Return validation accuracy as the optimization objective
    val_accuracy = history.history['val_accuracy'][-1]
```

```

    return val_accuracy

# Create a study object and optimize the objective function
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50) # You can set n_trials as per your
↳ requirement

# Get the best hyperparameters found by Optuna
best_params = study.best_params
print("Best hyperparameters:", best_params)

```

```

[I 2025-03-25 11:20:08,725] A new study created in memory with name: no-
name-0b8034e0-d6ad-4f22-8f42-a5aa9be4c639
[I 2025-03-25 11:20:22,214] Trial 0 finished with value: 0.8677419424057007 and
parameters: {'num_layers': 1, 'units': 128, 'activation_function': 'leaky_relu',
'learning_rate': 0.0352371432367099, 'batch_size': 96, 'optimizer': 'sgd',
'epochs': 85, 'dropout_rate': 0.34994771839276667}. Best is trial 0 with value:
0.8677419424057007.
[I 2025-03-25 11:20:35,890] Trial 1 finished with value: 0.8677419424057007 and
parameters: {'num_layers': 7, 'units': 112, 'activation_function': 'relu',
'learning_rate': 0.0003873208526488622, 'batch_size': 128, 'optimizer': 'sgd',
'epochs': 71, 'dropout_rate': 0.395438038568674}. Best is trial 0 with value:
0.8677419424057007.
[I 2025-03-25 11:20:41,891] Trial 2 finished with value: 0.8677419424057007 and
parameters: {'num_layers': 7, 'units': 128, 'activation_function': 'leaky_relu',
'learning_rate': 1.054285751180361e-05, 'batch_size': 64, 'optimizer':
'rmsprop', 'epochs': 20, 'dropout_rate': 0.21144943285465342}. Best is trial 0
with value: 0.8677419424057007.
[I 2025-03-25 11:20:53,027] Trial 3 finished with value: 0.8677419424057007 and
parameters: {'num_layers': 5, 'units': 128, 'activation_function': 'tanh',
'learning_rate': 0.0022348446254259937, 'batch_size': 128, 'optimizer': 'adam',
'epochs': 51, 'dropout_rate': 0.15298613166250272}. Best is trial 0 with value:
0.8677419424057007.
[I 2025-03-25 11:21:10,008] Trial 4 finished with value: 0.8677419424057007 and
parameters: {'num_layers': 5, 'units': 112, 'activation_function': 'relu',
'learning_rate': 0.0031194211334162397, 'batch_size': 32, 'optimizer': 'adam',
'epochs': 64, 'dropout_rate': 0.3411882100585635}. Best is trial 0 with value:
0.8677419424057007.
[I 2025-03-25 11:21:14,672] Trial 5 finished with value: 0.8677419424057007 and
parameters: {'num_layers': 4, 'units': 64, 'activation_function': 'tanh',
'learning_rate': 0.003990515256194739, 'batch_size': 32, 'optimizer': 'rmsprop',
'epochs': 15, 'dropout_rate': 0.4286733504796898}. Best is trial 0 with value:
0.8677419424057007.
[I 2025-03-25 11:21:30,113] Trial 6 finished with value: 0.8677419424057007 and
parameters: {'num_layers': 7, 'units': 48, 'activation_function': 'leaky_relu',
'learning_rate': 0.08141339578416591, 'batch_size': 96, 'optimizer': 'sgd',
'epochs': 88, 'dropout_rate': 0.17656469833083524}. Best is trial 0 with value:

```

0.8677419424057007.

[I 2025-03-25 11:21:33,585] Trial 7 finished with value: 0.8677419424057007 and parameters: {'num_layers': 1, 'units': 32, 'activation_function': 'sigmoid', 'learning_rate': 0.0009917991402760458, 'batch_size': 32, 'optimizer': 'rmsprop', 'epochs': 14, 'dropout_rate': 0.1650951474644446}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:21:38,080] Trial 8 finished with value: 0.8677419424057007 and parameters: {'num_layers': 6, 'units': 32, 'activation_function': 'relu', 'learning_rate': 0.041923328773665086, 'batch_size': 64, 'optimizer': 'sgd', 'epochs': 19, 'dropout_rate': 0.2767747370828906}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:21:41,438] Trial 9 finished with value: 0.8677419424057007 and parameters: {'num_layers': 4, 'units': 32, 'activation_function': 'tanh', 'learning_rate': 0.00876178146937632, 'batch_size': 128, 'optimizer': 'rmsprop', 'epochs': 13, 'dropout_rate': 0.3699716420495218}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:21:57,252] Trial 10 finished with value: 0.8677419424057007 and parameters: {'num_layers': 1, 'units': 96, 'activation_function': 'leaky_relu', 'learning_rate': 0.00015102674197413666, 'batch_size': 96, 'optimizer': 'sgd', 'epochs': 96, 'dropout_rate': 0.4845438354015498}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:22:08,514] Trial 11 finished with value: 0.8677419424057007 and parameters: {'num_layers': 2, 'units': 96, 'activation_function': 'relu', 'learning_rate': 0.00017518703307305234, 'batch_size': 128, 'optimizer': 'sgd', 'epochs': 70, 'dropout_rate': 0.4003308395089937}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:22:21,608] Trial 12 finished with value: 0.8677419424057007 and parameters: {'num_layers': 3, 'units': 112, 'activation_function': 'sigmoid', 'learning_rate': 0.00024124662154970905, 'batch_size': 96, 'optimizer': 'sgd', 'epochs': 76, 'dropout_rate': 0.2917489911140931}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:22:30,007] Trial 13 finished with value: 0.8677419424057007 and parameters: {'num_layers': 3, 'units': 80, 'activation_function': 'relu', 'learning_rate': 0.02006719859220909, 'batch_size': 128, 'optimizer': 'sgd', 'epochs': 50, 'dropout_rate': 0.46101445809432523}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:22:43,509] Trial 14 finished with value: 0.8677419424057007 and parameters: {'num_layers': 2, 'units': 112, 'activation_function': 'leaky_relu', 'learning_rate': 2.3814806318575374e-05, 'batch_size': 96, 'optimizer': 'sgd', 'epochs': 83, 'dropout_rate': 0.31898277698923266}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:22:52,162] Trial 15 finished with value: 0.8677419424057007 and parameters: {'num_layers': 6, 'units': 128, 'activation_function': 'relu', 'learning_rate': 0.00045679930261652074, 'batch_size': 64, 'optimizer': 'sgd', 'epochs': 39, 'dropout_rate': 0.25207581272187285}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:23:08,624] Trial 16 finished with value: 0.8677419424057007 and parameters: {'num_layers': 3, 'units': 96, 'activation_function': 'leaky_relu',

'learning_rate': 5.0704290176559784e-05, 'batch_size': 128, 'optimizer': 'adam', 'epochs': 99, 'dropout_rate': 0.38240933114923986}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:23:20,006] Trial 17 finished with value: 0.8677419424057007 and parameters: {'num_layers': 6, 'units': 80, 'activation_function': 'sigmoid', 'learning_rate': 0.0010032032172433637, 'batch_size': 96, 'optimizer': 'sgd', 'epochs': 62, 'dropout_rate': 0.4275221416327326}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:23:34,203] Trial 18 finished with value: 0.8677419424057007 and parameters: {'num_layers': 5, 'units': 112, 'activation_function': 'leaky_relu', 'learning_rate': 0.011816523728572345, 'batch_size': 128, 'optimizer': 'sgd', 'epochs': 80, 'dropout_rate': 0.3501389981980293}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:23:41,240] Trial 19 finished with value: 0.8677419424057007 and parameters: {'num_layers': 2, 'units': 128, 'activation_function': 'relu', 'learning_rate': 4.982949458339582e-05, 'batch_size': 96, 'optimizer': 'adam', 'epochs': 36, 'dropout_rate': 0.23238724838654345}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:23:58,999] Trial 20 finished with value: 0.8677419424057007 and parameters: {'num_layers': 7, 'units': 64, 'activation_function': 'leaky_relu', 'learning_rate': 0.09714554492198034, 'batch_size': 64, 'optimizer': 'sgd', 'epochs': 92, 'dropout_rate': 0.4962396592871778}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:24:08,129] Trial 21 finished with value: 0.8677419424057007 and parameters: {'num_layers': 7, 'units': 128, 'activation_function': 'leaky_relu', 'learning_rate': 1.2820248620835342e-05, 'batch_size': 64, 'optimizer': 'rmsprop', 'epochs': 32, 'dropout_rate': 0.1976904628874311}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:24:23,663] Trial 22 finished with value: 0.8677419424057007 and parameters: {'num_layers': 6, 'units': 128, 'activation_function': 'leaky_relu', 'learning_rate': 5.2672373146359026e-05, 'batch_size': 64, 'optimizer': 'rmsprop', 'epochs': 70, 'dropout_rate': 0.1300669160852879}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:24:37,124] Trial 23 finished with value: 0.8677419424057007 and parameters: {'num_layers': 7, 'units': 112, 'activation_function': 'leaky_relu', 'learning_rate': 1.106612487844347e-05, 'batch_size': 64, 'optimizer': 'rmsprop', 'epochs': 59, 'dropout_rate': 0.22151347490937226}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:24:42,812] Trial 24 finished with value: 0.8677419424057007 and parameters: {'num_layers': 5, 'units': 96, 'activation_function': 'leaky_relu', 'learning_rate': 0.0005671694151083727, 'batch_size': 96, 'optimizer': 'rmsprop', 'epochs': 25, 'dropout_rate': 0.10381149685943017}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:24:54,127] Trial 25 finished with value: 0.8677419424057007 and parameters: {'num_layers': 7, 'units': 128, 'activation_function': 'relu', 'learning_rate': 0.005743598069340284, 'batch_size': 64, 'optimizer': 'rmsprop', 'epochs': 47, 'dropout_rate': 0.3259192035717117}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:25:07,740] Trial 26 finished with value: 0.8677419424057007 and parameters: {'num_layers': 6, 'units': 112, 'activation_function': 'tanh', 'learning_rate': 0.03288975573940109, 'batch_size': 96, 'optimizer': 'sgd', 'epochs': 73, 'dropout_rate': 0.2736380776814926}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:25:20,669] Trial 27 finished with value: 0.8677419424057007 and parameters: {'num_layers': 1, 'units': 128, 'activation_function': 'sigmoid', 'learning_rate': 0.00011322607177332031, 'batch_size': 128, 'optimizer': 'rmsprop', 'epochs': 86, 'dropout_rate': 0.4170655367430546}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:25:32,432] Trial 28 finished with value: 0.8677419424057007 and parameters: {'num_layers': 4, 'units': 112, 'activation_function': 'leaky_relu', 'learning_rate': 0.001918735058497413, 'batch_size': 32, 'optimizer': 'adam', 'epochs': 44, 'dropout_rate': 0.3057203575685503}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:25:42,368] Trial 29 finished with value: 0.8677419424057007 and parameters: {'num_layers': 5, 'units': 128, 'activation_function': 'tanh', 'learning_rate': 2.355378037205325e-05, 'batch_size': 128, 'optimizer': 'sgd', 'epochs': 54, 'dropout_rate': 0.4549203460945441}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:25:57,684] Trial 30 finished with value: 0.8677419424057007 and parameters: {'num_layers': 7, 'units': 96, 'activation_function': 'relu', 'learning_rate': 0.0015327805208493774, 'batch_size': 96, 'optimizer': 'adam', 'epochs': 66, 'dropout_rate': 0.37333511660577157}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:26:13,116] Trial 31 finished with value: 0.8677419424057007 and parameters: {'num_layers': 5, 'units': 128, 'activation_function': 'tanh', 'learning_rate': 0.0029125937992094775, 'batch_size': 128, 'optimizer': 'adam', 'epochs': 78, 'dropout_rate': 0.16102035471437234}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:26:20,249] Trial 32 finished with value: 0.8677419424057007 and parameters: {'num_layers': 6, 'units': 128, 'activation_function': 'tanh', 'learning_rate': 0.0003610778901031674, 'batch_size': 128, 'optimizer': 'adam', 'epochs': 26, 'dropout_rate': 0.13325569184728336}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:26:35,400] Trial 33 finished with value: 0.13225805759429932 and parameters: {'num_layers': 5, 'units': 112, 'activation_function': 'tanh', 'learning_rate': 0.0033670324713933894, 'batch_size': 32, 'optimizer': 'adam', 'epochs': 56, 'dropout_rate': 0.2064447066417679}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:26:51,106] Trial 34 finished with value: 0.8677419424057007 and parameters: {'num_layers': 3, 'units': 128, 'activation_function': 'tanh', 'learning_rate': 8.52790905061704e-05, 'batch_size': 128, 'optimizer': 'adam', 'epochs': 90, 'dropout_rate': 0.189455068119563}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:27:02,747] Trial 35 finished with value: 0.8677419424057007 and parameters: {'num_layers': 4, 'units': 112, 'activation_function': 'relu', 'learning_rate': 0.051100093039042546, 'batch_size': 128, 'optimizer':

'rmsprop', 'epochs': 64, 'dropout_rate': 0.148172646016138}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:27:08,769] Trial 36 finished with value: 0.8677419424057007 and parameters: {'num_layers': 6, 'units': 64, 'activation_function': 'leaky_relu', 'learning_rate': 0.02177555621009844, 'batch_size': 96, 'optimizer': 'adam', 'epochs': 21, 'dropout_rate': 0.24852093281819926}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:27:30,469] Trial 37 finished with value: 0.8677419424057007 and parameters: {'num_layers': 7, 'units': 80, 'activation_function': 'tanh', 'learning_rate': 0.006488158018852527, 'batch_size': 32, 'optimizer': 'sgd', 'epochs': 84, 'dropout_rate': 0.35002892318529133}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:27:34,412] Trial 38 finished with value: 0.8677419424057007 and parameters: {'num_layers': 7, 'units': 48, 'activation_function': 'sigmoid', 'learning_rate': 0.0007299830539634195, 'batch_size': 64, 'optimizer': 'rmsprop', 'epochs': 10, 'dropout_rate': 0.4053502623516279}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:27:48,937] Trial 39 finished with value: 0.8677419424057007 and parameters: {'num_layers': 1, 'units': 112, 'activation_function': 'leaky_relu', 'learning_rate': 0.0022072833578056007, 'batch_size': 96, 'optimizer': 'sgd', 'epochs': 94, 'dropout_rate': 0.18042880477686904}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:28:00,243] Trial 40 finished with value: 0.8677419424057007 and parameters: {'num_layers': 4, 'units': 96, 'activation_function': 'relu', 'learning_rate': 0.010978535474207043, 'batch_size': 128, 'optimizer': 'sgd', 'epochs': 68, 'dropout_rate': 0.1030058172079873}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:28:14,526] Trial 41 finished with value: 0.8677419424057007 and parameters: {'num_layers': 5, 'units': 128, 'activation_function': 'relu', 'learning_rate': 0.0012857537116692742, 'batch_size': 32, 'optimizer': 'adam', 'epochs': 51, 'dropout_rate': 0.34136140263790193}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:28:34,649] Trial 42 finished with value: 0.8677419424057007 and parameters: {'num_layers': 6, 'units': 112, 'activation_function': 'relu', 'learning_rate': 0.004914117027761698, 'batch_size': 32, 'optimizer': 'adam', 'epochs': 74, 'dropout_rate': 0.3883562974208982}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:28:48,707] Trial 43 finished with value: 0.8677419424057007 and parameters: {'num_layers': 2, 'units': 128, 'activation_function': 'relu', 'learning_rate': 0.00027271262447581555, 'batch_size': 32, 'optimizer': 'adam', 'epochs': 61, 'dropout_rate': 0.29037380467713164}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:28:52,452] Trial 44 finished with value: 0.8677419424057007 and parameters: {'num_layers': 3, 'units': 112, 'activation_function': 'relu', 'learning_rate': 0.01897229171538708, 'batch_size': 64, 'optimizer': 'sgd', 'epochs': 17, 'dropout_rate': 0.44727236654402963}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:29:00,891] Trial 45 finished with value: 0.8677419424057007 and

parameters: {'num_layers': 4, 'units': 96, 'activation_function': 'leaky_relu', 'learning_rate': 0.0008634653004771903, 'batch_size': 96, 'optimizer': 'adam', 'epochs': 40, 'dropout_rate': 0.3413078329934144}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:29:13,469] Trial 46 finished with value: 0.8677419424057007 and parameters: {'num_layers': 5, 'units': 128, 'activation_function': 'relu', 'learning_rate': 0.06417718833798251, 'batch_size': 128, 'optimizer': 'sgd', 'epochs': 71, 'dropout_rate': 0.36182690209301166}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:29:30,353] Trial 47 finished with value: 0.8677419424057007 and parameters: {'num_layers': 6, 'units': 112, 'activation_function': 'sigmoid', 'learning_rate': 0.032900751125760276, 'batch_size': 64, 'optimizer': 'rmsprop', 'epochs': 81, 'dropout_rate': 0.31576528227686546}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:29:49,148] Trial 48 finished with value: 0.8677419424057007 and parameters: {'num_layers': 6, 'units': 128, 'activation_function': 'tanh', 'learning_rate': 0.014824459711025225, 'batch_size': 32, 'optimizer': 'sgd', 'epochs': 76, 'dropout_rate': 0.2691987617054194}. Best is trial 0 with value: 0.8677419424057007.

[I 2025-03-25 11:30:03,465] Trial 49 finished with value: 0.8677419424057007 and parameters: {'num_layers': 3, 'units': 48, 'activation_function': 'leaky_relu', 'learning_rate': 0.0028707829806297255, 'batch_size': 128, 'optimizer': 'adam', 'epochs': 88, 'dropout_rate': 0.39197860147555835}. Best is trial 0 with value: 0.8677419424057007.

Best hyperparameters: {'num_layers': 1, 'units': 128, 'activation_function': 'leaky_relu', 'learning_rate': 0.0352371432367099, 'batch_size': 96, 'optimizer': 'sgd', 'epochs': 85, 'dropout_rate': 0.34994771839276667}

```
[66]: from optuna.visualization import plot_optimization_history, \
      ↪ plot_parallel_coordinate, plot_slice, plot_contour, plot_param_importances
      plot_optimization_history(study).show()
```

```
[67]: import optuna.visualization as vis

      # Plot the optimization history
      vis.plot_optimization_history(study)

      # Plot the hyperparameter importance
      vis.plot_param_importances(study)
```

```
[68]: # Assuming you have already run Optuna and obtained the best parameters
      best_params = study.best_params

      # Create and compile the model with the best hyperparameters
      model = Sequential()

      # First hidden layer
```

```

model.add(Dense(units=best_params['units'],
                activation=best_params['activation_function'],
                input_dim=X_train.shape[1])) # Use the input dimension of your
↳ training data
model.add(Dropout(best_params['dropout_rate'])) # Apply dropout

# Add additional hidden layers
for _ in range(best_params['num_layers'] - 1): # Subtract 1 because we already
↳ added one layer above
    model.add(Dense(units=best_params['units'],
                    activation=best_params['activation_function']))
    model.add(Dropout(best_params['dropout_rate'])) # Apply dropout after each
↳ hidden layer

# Output layer (binary classification)
model.add(Dense(1, activation='sigmoid')) # Sigmoid activation for binary
↳ classification

# Select optimizer based on the best parameter
if best_params["optimizer"] == "adam":
    optimizer = Adam(learning_rate=best_params["learning_rate"])
elif best_params["optimizer"] == "sgd":
    optimizer = SGD(learning_rate=best_params["learning_rate"])
elif best_params["optimizer"] == "rmsprop":
    optimizer = RMSprop(learning_rate=best_params["learning_rate"])

# Compile the model
model.compile(optimizer=optimizer, loss="binary_crossentropy",
↳ metrics=["accuracy"])

# Train the model with the best hyperparameters
history = model.fit(X_train, y_train,
                    epochs=best_params["epochs"],
                    batch_size=best_params["batch_size"],
                    validation_data=(X_test, y_test),
                    verbose=1) # Set verbose=1 for detailed training logs

```

```

Epoch 1/85
13/13          1s 21ms/step -
accuracy: 0.5215 - loss: 0.7760 - val_accuracy: 0.8677 - val_loss: 3038.5225
Epoch 2/85
13/13          0s 9ms/step -
accuracy: 0.8186 - loss: 0.5097 - val_accuracy: 0.8677 - val_loss: 4522.1348
Epoch 3/85
13/13          0s 10ms/step -
accuracy: 0.8762 - loss: 0.4209 - val_accuracy: 0.8677 - val_loss: 5116.9458
Epoch 4/85

```

13/13 0s 9ms/step -
 accuracy: 0.8837 - loss: 0.3969 - val_accuracy: 0.8677 - val_loss: 5472.4248
 Epoch 5/85
 13/13 0s 9ms/step -
 accuracy: 0.8912 - loss: 0.3578 - val_accuracy: 0.8677 - val_loss: 5612.2144
 Epoch 6/85
 13/13 0s 9ms/step -
 accuracy: 0.8906 - loss: 0.3524 - val_accuracy: 0.8677 - val_loss: 5716.4277
 Epoch 7/85
 13/13 0s 10ms/step -
 accuracy: 0.8904 - loss: 0.3468 - val_accuracy: 0.8677 - val_loss: 5711.4204
 Epoch 8/85
 13/13 0s 9ms/step -
 accuracy: 0.8874 - loss: 0.3603 - val_accuracy: 0.8677 - val_loss: 5738.8291
 Epoch 9/85
 13/13 0s 10ms/step -
 accuracy: 0.8749 - loss: 0.3756 - val_accuracy: 0.8677 - val_loss: 5780.1060
 Epoch 10/85
 13/13 0s 9ms/step -
 accuracy: 0.9032 - loss: 0.3174 - val_accuracy: 0.8677 - val_loss: 5780.6416
 Epoch 11/85
 13/13 0s 10ms/step -
 accuracy: 0.8957 - loss: 0.3325 - val_accuracy: 0.8677 - val_loss: 5707.2148
 Epoch 12/85
 13/13 0s 9ms/step -
 accuracy: 0.8958 - loss: 0.3252 - val_accuracy: 0.8677 - val_loss: 5631.1074
 Epoch 13/85
 13/13 0s 10ms/step -
 accuracy: 0.8972 - loss: 0.3181 - val_accuracy: 0.8677 - val_loss: 5524.9932
 Epoch 14/85
 13/13 0s 9ms/step -
 accuracy: 0.8865 - loss: 0.3414 - val_accuracy: 0.8677 - val_loss: 5471.2739
 Epoch 15/85
 13/13 0s 10ms/step -
 accuracy: 0.8853 - loss: 0.3442 - val_accuracy: 0.8677 - val_loss: 5402.9604
 Epoch 16/85
 13/13 0s 9ms/step -
 accuracy: 0.8866 - loss: 0.3476 - val_accuracy: 0.8677 - val_loss: 5363.3252
 Epoch 17/85
 13/13 0s 10ms/step -
 accuracy: 0.8962 - loss: 0.3259 - val_accuracy: 0.8677 - val_loss: 5316.3691
 Epoch 18/85
 13/13 0s 9ms/step -
 accuracy: 0.8913 - loss: 0.3243 - val_accuracy: 0.8677 - val_loss: 5198.5747
 Epoch 19/85
 13/13 0s 10ms/step -
 accuracy: 0.8934 - loss: 0.3214 - val_accuracy: 0.8677 - val_loss: 5224.7607
 Epoch 20/85

13/13 0s 9ms/step -
 accuracy: 0.8902 - loss: 0.3272 - val_accuracy: 0.8677 - val_loss: 5188.1724
 Epoch 21/85
 13/13 0s 9ms/step -
 accuracy: 0.8963 - loss: 0.3120 - val_accuracy: 0.8677 - val_loss: 5114.4199
 Epoch 22/85
 13/13 0s 9ms/step -
 accuracy: 0.9021 - loss: 0.3011 - val_accuracy: 0.8677 - val_loss: 5093.1592
 Epoch 23/85
 13/13 0s 10ms/step -
 accuracy: 0.8857 - loss: 0.3342 - val_accuracy: 0.8677 - val_loss: 4976.6611
 Epoch 24/85
 13/13 0s 9ms/step -
 accuracy: 0.8915 - loss: 0.3219 - val_accuracy: 0.8677 - val_loss: 4900.4019
 Epoch 25/85
 13/13 0s 10ms/step -
 accuracy: 0.8946 - loss: 0.3123 - val_accuracy: 0.8677 - val_loss: 4855.6929
 Epoch 26/85
 13/13 0s 9ms/step -
 accuracy: 0.8918 - loss: 0.3289 - val_accuracy: 0.8677 - val_loss: 4833.7261
 Epoch 27/85
 13/13 0s 10ms/step -
 accuracy: 0.8818 - loss: 0.3466 - val_accuracy: 0.8677 - val_loss: 4876.9287
 Epoch 28/85
 13/13 0s 9ms/step -
 accuracy: 0.8796 - loss: 0.3527 - val_accuracy: 0.8677 - val_loss: 4934.4419
 Epoch 29/85
 13/13 0s 10ms/step -
 accuracy: 0.8984 - loss: 0.3138 - val_accuracy: 0.8677 - val_loss: 4838.9116
 Epoch 30/85
 13/13 0s 9ms/step -
 accuracy: 0.8978 - loss: 0.3096 - val_accuracy: 0.8677 - val_loss: 4832.9731
 Epoch 31/85
 13/13 0s 10ms/step -
 accuracy: 0.8883 - loss: 0.3351 - val_accuracy: 0.8677 - val_loss: 4830.7124
 Epoch 32/85
 13/13 0s 9ms/step -
 accuracy: 0.8902 - loss: 0.3220 - val_accuracy: 0.8677 - val_loss: 4829.0850
 Epoch 33/85
 13/13 0s 9ms/step -
 accuracy: 0.8914 - loss: 0.3118 - val_accuracy: 0.8677 - val_loss: 4790.7168
 Epoch 34/85
 13/13 0s 9ms/step -
 accuracy: 0.8907 - loss: 0.3364 - val_accuracy: 0.8677 - val_loss: 4752.2388
 Epoch 35/85
 13/13 0s 10ms/step -
 accuracy: 0.8963 - loss: 0.3152 - val_accuracy: 0.8677 - val_loss: 4692.7695
 Epoch 36/85

13/13 0s 9ms/step -
 accuracy: 0.8893 - loss: 0.3254 - val_accuracy: 0.8677 - val_loss: 4625.4233
 Epoch 37/85
 13/13 0s 10ms/step -
 accuracy: 0.8878 - loss: 0.3317 - val_accuracy: 0.8677 - val_loss: 4579.3276
 Epoch 38/85
 13/13 0s 9ms/step -
 accuracy: 0.8980 - loss: 0.3239 - val_accuracy: 0.8677 - val_loss: 4452.7637
 Epoch 39/85
 13/13 0s 10ms/step -
 accuracy: 0.8779 - loss: 0.3422 - val_accuracy: 0.8677 - val_loss: 4421.5679
 Epoch 40/85
 13/13 0s 9ms/step -
 accuracy: 0.8946 - loss: 0.3133 - val_accuracy: 0.8677 - val_loss: 4384.0063
 Epoch 41/85
 13/13 0s 10ms/step -
 accuracy: 0.8931 - loss: 0.3159 - val_accuracy: 0.8677 - val_loss: 4349.4692
 Epoch 42/85
 13/13 0s 9ms/step -
 accuracy: 0.8907 - loss: 0.3147 - val_accuracy: 0.8677 - val_loss: 4321.9688
 Epoch 43/85
 13/13 0s 9ms/step -
 accuracy: 0.8999 - loss: 0.3009 - val_accuracy: 0.8677 - val_loss: 4327.8481
 Epoch 44/85
 13/13 0s 9ms/step -
 accuracy: 0.8934 - loss: 0.3136 - val_accuracy: 0.8677 - val_loss: 4312.3320
 Epoch 45/85
 13/13 0s 10ms/step -
 accuracy: 0.8765 - loss: 0.3456 - val_accuracy: 0.8677 - val_loss: 4295.6782
 Epoch 46/85
 13/13 0s 9ms/step -
 accuracy: 0.8940 - loss: 0.3238 - val_accuracy: 0.8677 - val_loss: 4231.4790
 Epoch 47/85
 13/13 0s 9ms/step -
 accuracy: 0.8902 - loss: 0.3197 - val_accuracy: 0.8677 - val_loss: 4215.0742
 Epoch 48/85
 13/13 0s 9ms/step -
 accuracy: 0.8960 - loss: 0.3057 - val_accuracy: 0.8677 - val_loss: 4198.1235
 Epoch 49/85
 13/13 0s 9ms/step -
 accuracy: 0.8933 - loss: 0.3242 - val_accuracy: 0.8677 - val_loss: 4236.0728
 Epoch 50/85
 13/13 0s 9ms/step -
 accuracy: 0.8854 - loss: 0.3287 - val_accuracy: 0.8677 - val_loss: 4234.9009
 Epoch 51/85
 13/13 0s 9ms/step -
 accuracy: 0.8756 - loss: 0.3643 - val_accuracy: 0.8677 - val_loss: 4229.2192
 Epoch 52/85

13/13 0s 9ms/step -
 accuracy: 0.8947 - loss: 0.3186 - val_accuracy: 0.8677 - val_loss: 4200.7754
 Epoch 53/85
 13/13 0s 10ms/step -
 accuracy: 0.8930 - loss: 0.3209 - val_accuracy: 0.8677 - val_loss: 4224.4316
 Epoch 54/85
 13/13 0s 9ms/step -
 accuracy: 0.9036 - loss: 0.2997 - val_accuracy: 0.8677 - val_loss: 4191.8379
 Epoch 55/85
 13/13 0s 10ms/step -
 accuracy: 0.8876 - loss: 0.3327 - val_accuracy: 0.8677 - val_loss: 4235.3281
 Epoch 56/85
 13/13 0s 9ms/step -
 accuracy: 0.8973 - loss: 0.3120 - val_accuracy: 0.8677 - val_loss: 4174.3970
 Epoch 57/85
 13/13 0s 9ms/step -
 accuracy: 0.8917 - loss: 0.3101 - val_accuracy: 0.8677 - val_loss: 4227.1021
 Epoch 58/85
 13/13 0s 9ms/step -
 accuracy: 0.8876 - loss: 0.3300 - val_accuracy: 0.8677 - val_loss: 4223.6528
 Epoch 59/85
 13/13 0s 10ms/step -
 accuracy: 0.8866 - loss: 0.3344 - val_accuracy: 0.8677 - val_loss: 4265.0527
 Epoch 60/85
 13/13 0s 9ms/step -
 accuracy: 0.8921 - loss: 0.3122 - val_accuracy: 0.8677 - val_loss: 4281.8159
 Epoch 61/85
 13/13 0s 10ms/step -
 accuracy: 0.8992 - loss: 0.3010 - val_accuracy: 0.8677 - val_loss: 4304.6851
 Epoch 62/85
 13/13 0s 9ms/step -
 accuracy: 0.9101 - loss: 0.2869 - val_accuracy: 0.8677 - val_loss: 4318.4897
 Epoch 63/85
 13/13 0s 10ms/step -
 accuracy: 0.8945 - loss: 0.3186 - val_accuracy: 0.8677 - val_loss: 4289.4731
 Epoch 64/85
 13/13 0s 9ms/step -
 accuracy: 0.8926 - loss: 0.3223 - val_accuracy: 0.8677 - val_loss: 4269.9531
 Epoch 65/85
 13/13 0s 9ms/step -
 accuracy: 0.8902 - loss: 0.3126 - val_accuracy: 0.8677 - val_loss: 4212.1396
 Epoch 66/85
 13/13 0s 9ms/step -
 accuracy: 0.8872 - loss: 0.3234 - val_accuracy: 0.8677 - val_loss: 4237.6333
 Epoch 67/85
 13/13 0s 9ms/step -
 accuracy: 0.8832 - loss: 0.3335 - val_accuracy: 0.8677 - val_loss: 4271.4990
 Epoch 68/85

13/13 0s 9ms/step -
 accuracy: 0.8967 - loss: 0.3077 - val_accuracy: 0.8677 - val_loss: 4296.5107
 Epoch 69/85
 13/13 0s 9ms/step -
 accuracy: 0.8790 - loss: 0.3342 - val_accuracy: 0.8677 - val_loss: 4323.4707
 Epoch 70/85
 13/13 0s 9ms/step -
 accuracy: 0.8841 - loss: 0.3466 - val_accuracy: 0.8677 - val_loss: 4295.8154
 Epoch 71/85
 13/13 0s 10ms/step -
 accuracy: 0.8753 - loss: 0.3475 - val_accuracy: 0.8677 - val_loss: 4235.2188
 Epoch 72/85
 13/13 0s 9ms/step -
 accuracy: 0.8896 - loss: 0.3053 - val_accuracy: 0.8677 - val_loss: 4303.0547
 Epoch 73/85
 13/13 0s 9ms/step -
 accuracy: 0.8788 - loss: 0.3356 - val_accuracy: 0.8677 - val_loss: 4325.4355
 Epoch 74/85
 13/13 0s 9ms/step -
 accuracy: 0.8905 - loss: 0.3296 - val_accuracy: 0.8677 - val_loss: 4308.0825
 Epoch 75/85
 13/13 0s 9ms/step -
 accuracy: 0.8871 - loss: 0.3275 - val_accuracy: 0.8677 - val_loss: 4270.3301
 Epoch 76/85
 13/13 0s 9ms/step -
 accuracy: 0.9040 - loss: 0.2998 - val_accuracy: 0.8677 - val_loss: 4273.1562
 Epoch 77/85
 13/13 0s 10ms/step -
 accuracy: 0.8971 - loss: 0.3092 - val_accuracy: 0.8677 - val_loss: 4251.8315
 Epoch 78/85
 13/13 0s 9ms/step -
 accuracy: 0.9061 - loss: 0.2808 - val_accuracy: 0.8677 - val_loss: 4212.7832
 Epoch 79/85
 13/13 0s 9ms/step -
 accuracy: 0.8964 - loss: 0.3013 - val_accuracy: 0.8677 - val_loss: 4186.1108
 Epoch 80/85
 13/13 0s 9ms/step -
 accuracy: 0.8846 - loss: 0.3186 - val_accuracy: 0.8677 - val_loss: 4182.9321
 Epoch 81/85
 13/13 0s 9ms/step -
 accuracy: 0.9063 - loss: 0.2846 - val_accuracy: 0.8677 - val_loss: 4204.7061
 Epoch 82/85
 13/13 0s 9ms/step -
 accuracy: 0.8988 - loss: 0.3032 - val_accuracy: 0.8677 - val_loss: 4221.3589
 Epoch 83/85
 13/13 0s 10ms/step -
 accuracy: 0.8864 - loss: 0.3307 - val_accuracy: 0.8677 - val_loss: 4180.0796
 Epoch 84/85

```

13/13          0s 9ms/step -
accuracy: 0.9012 - loss: 0.3011 - val_accuracy: 0.8677 - val_loss: 4168.3130
Epoch 85/85
13/13          0s 10ms/step -
accuracy: 0.8750 - loss: 0.3497 - val_accuracy: 0.8677 - val_loss: 4242.0947

```

```

[70]: # Evaluate the model on test data (optional but recommended)
train_loss, ann_train_accuracy = model.evaluate(X_train, y_train, verbose=0)

print(f"Train accuracy: {ann_train_accuracy:.4f}")
print(f"Train loss: {train_loss:.4f}")

test_loss, ann_test_accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {ann_test_accuracy:.4f}")
print(f"Test loss: {test_loss:.4f}")

```

```

Train accuracy: 0.8926
Train loss: 0.3011
Test accuracy: 0.8677
Test loss: 4242.0947

```

2.1 Model Selection

```

[71]: final_data = pd.DataFrame({"Models": ['Logistic', 'SVM', "DT", "RF", "XGBOOST",
↪ "ANN_Classifier"],
                                "TRAIN_ACC_SCORE":
↪ [log_train_accuracy, svm_train_accuracy, dt_train_accuracy, rf_train_accuracy, xgb_train_accuracy,
↪ ann_train_accuracy],
                                "TEST ACC_SCORE":
↪ [log_test_accuracy, svm_test_accuracy, dt_test_accuracy, rf_test_accuracy, xgb_test_accuracy,
↪ ann_test_accuracy],
                                "CV_SCORE": [log_cv, svm_cv, dt_cv, rf_cv, xgb_cv, ""]})

```

```

[72]: final_data

```

```

[72]:
      Models  TRAIN_ACC_SCORE  TEST ACC_SCORE  CV_SCORE
0   Logistic      89.983845      13.225806  86.451613
1         SVM      94.991922      86.774194  88.064516
2         DT      89.579968      86.774194  85.806452
3         RF      99.111470      86.774194  87.741935
4   XGBOOST      99.030695      86.129032  85.483871
5 ANN_Classifier      0.892569      0.867742

```

```

[ ]:

```